



UNIVERSITATEA DIN BUCUREȘTI

**FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

**CUANTIFICAREA SCHIMBĂRILOR
NETE DIN ACTUALIZĂRILE
JOCURILOR VIDEO FOLOSIND
PROCESAREA LIMBAJULUI NATURAL**

Absolvent

Ungureanu Matei-Ștefan

Coordonatori științifici

Prof. dr. Leuștean Ioana Gabriela

Asist. drd. Macovei Bogdan

București, iunie 2026

Rezumat

Tranziția jocurilor video *multiplayer* către modelul de operare continuă a generat un volum masiv de date nestructurate sub forma notelor de actualizare (*patch notes*), cu care toți jucătorii competitivi trebuie să țină pasul. Prezenta lucrare abordează problema accesibilității acestor informații pentru jucătorii care revin după perioade lungi de absență, și propune NetPatch, un sistem capabil să cuantifice schimbările nete *net changes* ale jocurilor. Metodologia se bazează pe tehnici de procesare a limbajului natural (NLP) pentru a extrage și structura datele de pe platforma Fandom, întreținută de comunitate. Stocarea datelor este realizată de o bază de date orientată pe grafuri (Neo4j) pentru interogarea rapidă a relațiilor fizice dintre noduri și o bază de date relațională (PostgreSQL) pentru gestionarea adnotărilor și a mecanismului de revizuire manuală (*human-in-the-loop*). Pentru a depăși limitările modelelor lingvistice standard în fața jargonului jocurilor video, aplicația web implementează un modul NER personalizat, bazat pe reguli. Prin metoda supervizării la distanță (*distant supervision*), datele extrase au generat un set de antrenare cu ajutorul căruia a fost dezvoltat un model NER statistic, bazat pe arhitectura Transformer (RoBERTa), compilat specific pentru jocul Overwatch. Prin evaluarea acestuia a fost demonstrată capacitatea de generalizare pe date noi, obținând un scor F1 de peste 87% pentru eroi și abilități, și 75% pentru attribute, astfel fiind confirmată viabilitatea aplicării tehnicilor NLP pe datele nestructurate aparținând jocurilor video. Aplicația rezultată are o arhitectură modulară și este distribuită *open-source*, facilitând dezvoltarea acesteia în continuare pentru a integra jocuri noi și a crea module NER noi.

Abstract

The multiplayer video games' transition to the continuous operation model has generated a massive volume of unstructured data in the form of patch notes, with which all competitive players must keep up. This thesis addresses the accessibility problem of this information for players returning after extended periods of absence and proposes NetPatch, a system capable of quantifying video game net changes. The methodology relies on Natural Language Processing (NLP) techniques to extract and structure data from the community maintained Fandom platform. Data storage is done by a graph database (Neo4j) for the rapid querying of physical relationships between nodes and a relational database (PostgreSQL) for managing annotations and the manual review mechanism (*human-in-the-loop*). To overcome the limitations of standard language models when faced with video game jargon, the web application implements a custom, rule-based NER module. Through distant supervision, the extracted data generated a training set which was used to develop a statistical NER model based on the Transformer architecture (RoBERTa), specifically packaged for the game Overwatch. Evaluating the model proved its capacity to generalize well to unseen data. It achieved an F1 score of over 87% for heroes and abilities, and 75% for attributes, confirming that NLP techniques can be successfully applied to unstructured video game texts. The resulting application features a modular, open-source architecture, making it easy to integrate new games and train additional NER modules in the future.

Cuprins

1	Introducere	4
2	Preliminarii	5
3	Descrierea aplicației	6
3.1	Arhitectura sistemului	7
3.1.1	Construirea dicționarelor de entități	8
3.1.2	Extracția datelor de pe Fandom	9
3.1.3	Procesarea schimbărilor	11
3.1.4	Analiza amănunțită	12
3.1.5	Integritatea informației	14
3.1.6	Studiu de caz: integrarea jocului Marvel Rivals	15
3.2	Graful de cunoștințe	16
3.3	Aplicația web	18
3.4	Modulul NER pentru Overwatch	19
3.4.1	Antrenarea modelului	19
3.4.2	Modulul NER instalabil	22
4	Concluzii	22
	Bibliografie	24
Anexa 1	Ghid de integrare a unui joc	25

1. Introducere

În ultimul timp, jocurile video multiplayer au început să adopte modelul numit *Games as a Service* (GaaS). Lansarea inițială a unui joc reprezintă doar un punct de plecare, titlurile fiind menținute relevante pe parcursul anilor prin actualizări periodice ce aduc conținut nou și modificări constante de echilibrare (*balance changes*) asupra mecanicilor și atributelor entităților din joc, cu scopul de a asigura o experiență de joc echilibrată. Toate aceste modificări sunt prezentate prin intermediul notelor de actualizare (patch notes). Cu toate acestea, numărul mare de schimbări rezultă într-o problemă pentru utilizatorii care iau o pauză de la joc. La revenire, un jucător este întâmpinat de zeci de actualizări separate pe care trebuie să le citească pentru a ajunge la curent cu starea jocului, ceea ce este inefficient și obositor.

Fiind afectat eu însumi de această problemă, voi dezvolta **NetPatch**, o aplicație bazată pe procesarea limbajului natural (NLP). Aceasta va analiza automat istoricul actualizărilor și le va arăta utilizatorilor într-un format simplu și clar exact ce s-a schimbat într-un interval de timp, calculând schimbările nete (*net changes*). Deoarece valorile atributelor pot fluctua, atât pozitiv cât și negativ, pe parcursul mai multor actualizări succesive, schimbarea netă reflectă exclusiv rezultatul final resimțit de jucător la revenire.

Aplicația se adresează atât jucătorilor cât și dezvoltatorilor, arhitectura fiind gândită să fie extensibilă.

Obiectivele principale ale lucrării sunt:

- Construirea unui pipeline automat pentru extragerea și procesarea datelor nestructurate de pe platforma Fandom.
- Utilizarea unei baze de date orientate pe grafuri pentru a stoca și interoga eficient relațiile dintre actualizări, entități și schimbări.
- Dezvoltarea unei interfețe web pentru afișarea schimbărilor nete dintr-un interval de timp.
- Antrenarea unui model NER (bazat pe arhitectura RoBERTa) folosind un set de date generat automat, și împachetarea acestuia într-un modul independent.

Proiectul va fi dezvoltat *open-source*.

- Codul sursă va putea fi găsit pe GitHub la adresa:

<https://github.com/mateiungureanu/netpatch>

- Aplicația web va fi publicată online la adresa: <https://netpatch.msung.dev>

Lucrarea este împărțită în patru capitole. După această introducere, Capitolul 2 va explica termenii tehnici și conceptele de bază folosite în domeniul jocurilor video. Capitolul 3 va detalia arhitectura sistemului, de la extragerea datelor și stocarea lor, până la interfața web și antrenarea modelului NER pentru jocul Overwatch. În final, Capitolul 4 va prezenta concluziile trase în urma dezvoltării sistemului și planurile pentru îmbunătățirea lui pe viitor.

2. Preliminarii

Încă din etapa de analiză a cerințelor, a devenit evident că un model standard de procesare a limbajului natural pentru recunoașterea entităților cu nume (NER) nu este fezabil, deoarece terminologia jocurilor video este bazată pe jargon. De exemplu, încercând să extrag entitățile din propoziția „Tracer’s Pulse Pistols Damage decreased from 6 to 5.75” folosind un model clasic NER precum *en_core_web_sm*, eroul Tracer a fost identificat în mod eronat ca fiind un produs (*PRODUCT*). Astfel, a fost necesară implementarea unui modul NER personalizat, capabil să recunoască nativ toate entitățile specifice unui joc. În plus, analizând structura notelor de actualizare, simpla parcurgere secvențială a textului pentru extragerea entităților s-a dovedit a fi ineficientă, deoarece adesea contextul este declarat separat, la nivel de subtitlu sau prin indentare. Soluția adoptată constă într-un mecanism de păstrare a contextului și alipire a acestuia la linia de text ce descrie schimbarea efectivă, înainte de extragerea entităților.

În ceea ce privește arhitectura de stocare, decizia a fost fundamentată pe natura datelor procesate. Observând că istoricul modificărilor unui joc formează în mod natural o rețea de entități interconectate, am optat pentru o bază de date orientată pe grafuri. Această paradigmă oferă posibilitatea de a defini multiple tipuri de relații semantice, făcând dezvoltarea interogărilor mult

mai intuitivă comparativ cu modelele relaționale clasice. Mai mult, timpul de interogare este exponențial mai mic. În loc de a executa operațiuni costisitoare de alăturare a tabelor (JOIN), motorul bazei de date parcurge direct legăturile fizice dintre noduri.

Având în vedere prevalența limbii engleze în mediul jocurilor video, termenii de specialitate relevanți vor fi explicați în cele ce urmează și folosiți ca atare în restul documentului:

- *Fandom*: platformă de tip wiki (menținută de comunitate) ce servește drept sursa datelor
- *patch notes*: multitudinea schimbărilor aduse unui joc printr-o actualizare
- *balance changes*: multitudinea schimbărilor aduse printr-o actualizare
- *bug fix*: repararea unei mecanici/comportament neintenționat
- *buff*: modificare pozitivă adusă unui atribut, ce sporește eficiența acestuia
- *nerf*: modificare negativă adusă unui atribut, ce diminuează eficiența acestuia

Pentru a atinge obiectivele propuse, sistemul se bazează pe un set divers de tehnologii. Dezvoltarea a fost realizată în limbajul Python, folosind framework-ul web Django [1] pentru orchestrarea scripturilor și expunerea interfeței grafice. Stocarea datelor folosește două baze de date: Neo4j [4] pentru stocarea istoricului și PostgreSQL [6] pentru gestionarea datelor relaționale auxiliare. Componenta de procesare a limbajului natural a fost construită în jurul bibliotecii spaCy [3], folosită pentru aplicarea regulilor de extragere, în timp ce pentru recunoașterea avansată a entităților a fost antrenat un model Transformer, RoBERTa, pe un set de date generat prin supervizare la distanță. Utilizarea și integrarea detaliată a acestor tehnologii în cadrul aplicației vor fi prezentate în capitolul următor.

3. Descrierea aplicației

Aplicația NetPatch implementează un pipeline care preia date nestructurate de pe Fandom, le procesează, le stochează și le adnotează automat. În următoarele secțiuni, voi prezenta arhitectura modulară a sistemului, graful de cunoștințe, aplicația/interfața web, mecanismul de

adnotare automată prin supervizare la distanță, și antrenarea unui model pe setul de date obținut și împachetarea acestuia într-un modul NER instalabil, luând inspirație de la un modul NER pentru mediul legal românesc [5].

3.1 Arhitectura sistemului

Arhitectura este formată din mai multe componente cheie, fiecare fiind responsabilă pentru o parte specifică a pipeline-ului. Datele sunt extrase brut de pe Fandom, sunt curățate și trecute printr-un filtru care elimină secțiunile irelevante, apoi sunt preprocesate pentru a asigura structurarea datelor. Textul este apoi procesat și parcurs linie cu linie, salvându-se contextul. La pasul următor, liniile sunt trecute printr-un filtru ce clasifică schimbările care nu necesită o analiză amănunțită. Liniilor rămase li se alipește contextul și sunt parsate. În urma clasificării schimbările sunt salvate, iar cele cărora nu le-a fost atribuit un tip sunt rutate către o coadă de revizuire manuală.

Pentru stocarea datelor, este folosită o arhitectură hibridă cu două baze de date: una orientată pe grafuri, oferită de Neo4j [4], și una relațională, oferită de PostgreSQL [6]. Baza de date pe grafuri găzduiește toate datele ce țin de actualizările jocurilor, creând efectiv un graf de cunoștințe (Secțiunea 3.2) al întregului istoric al jocurilor. Baza de date relațională găzduiește toate schimbările acceptate, coada pentru verificări manuale, setul de date pentru antrenare, și toate cuvintele cheie necesare procesării, parsării, și analizei, fără a avea relații între tabele.

Orchestrarea întregului flux de date, de la rularea scripturilor de extracție până la comunicarea cu cele două baze de date, este gestionată prin intermediul framework-ului web Django [1]. Acesta acționează ca nucleu al arhitecturii, oferind un mediu robust pentru execuția codului Python, și fundația necesară expunerii unei interfețe grafice de administrare și vizualizare.

Aplicația este proiectată pentru a suporta trei tipuri de utilizatori:

- Jucătorul (Consumatorul): Acesta este utilizatorul care intră pe site pentru a vedea schimbările nete petrecute în absența sa: alege jocul, introduce perioada în care a lipsit și primește imediat o listă structurată cu toate schimbările. Astfel, scapă de cititul manual al notelor de actualizare.

- Dezvoltatorul (Contribuitorul): Deoarece arhitectura este modulară, platforma poate suporta ușor jocuri noi. Un programator poate adăuga un joc pe platformă prin implementarea unor clase specifice și integrarea lor în codul sursă printr-un *pull request*. Mai multe detalii privind integrarea unui joc nou se află în Anexa 1.
- Operatorul uman (Administratorul): Are rolul de a menține integritatea bazei de date prin revizuirea schimbărilor. Acest tip de utilizator are acces la interfața de administrare nativă Django. Printr-un sistem de grupuri și permisiuni, administratorii pot revizui cozile de așteptare, pot aproba atributele deduse automat și pot atribui manual o clasificare liniilor problematice, folosind butoane și acțiuni predefinite direct din panoul de control.

Aplicația conține două jocuri: Overwatch și Marvel Rivals. În cele ce urmează, detaliile legate de implementare vor trata părțile comune împreună cu dezvoltările particulare jocului Overwatch.

3.1.1 Construirea dicționarelor de entități

Primul pas în integrarea unui joc este construirea dicționarelor de entități. Acestea sunt necesare pentru a putea detecta entitățile specifice jocului, deoarece un modul standard de NER (e.g. *en_core_web_sm*) nu le poate identifica, iar un modul pentru jocuri nu există.

Pentru a realiza asta, modulul de construire necesită un URL către pagina de Fandom de bază a jocului respectiv. Folosind funcții specifice ale API-ului MediaWiki [2] pentru colectarea membrilor unei categorii, programul colectează toți eroii (entitățile controlabile de jucători) și îi salvează. În plus, având în vedere că Overwatch aplică schimbări și la nivelul rolurilor și al subrolurilor, programul colectează aceste roluri și le salvează pe post de eroi.

Pentru a obține abilitățile, programul accesează secvențial paginile Fandom ale tuturor eroilor și le scanează după marcaje specifice de tip *wikitext* care indică faptul că urmează să fie numită o abilitate (e.g. „| ability_name = Blink”). În plus, având în vedere că abilitățile sunt frecvent acompaniate și uneori chiar înlocuite cu tipurile acestora, programul salvează și tipul abilităților, obținute tot prin identificarea marcajelor specifice. (e.g. „| ability_type = Ability”)

În final, atributele ce se aplică și eroilor și abilităților sunt preluate dintr-un fișier static ce conține toate atributele cunoscute la momentul respectiv. Atributele noi sunt deduse inițial de

către modulul de analiză și sunt salvate în baza de date relațională cu un indicator de sincronizare inactiv. În urma revizuirii și aprobării acestora de către operatorul uman prin interfața web, sistemul declanșează automat un commit către GitHub, actualizând fișierul centralizat. Astfel, la următoarea execuție, constructorul de dicționare beneficiază de noile atribute validate.

Pe lângă dicționarul principal descris mai sus, pentru fiecare joc sunt create câteva dicționare secundare: un dicționarul ce mapează fiecare erou la un rol, și încă unul ce mapează fiecare abilitate la eroul de care face parte. Dacă un joc necesită resurse adiționale, programul poate fi extins. Astfel, pentru Overwatch este creat și un dicționar de tipuri, ce mapează fiecare abilitate la tipul acesteia (e.g. *Pulse Pistols* are tipul Primary Fire, *Blink* are tipul Ability, *Pulse Bomb* are tipul Ultimate).

Dicționarul principal	Dicționarul de roluri
<pre>{ "label": "HERO", "pattern": "Tracer" }, { "label": "ABILITY", "pattern": "Pulse Pistols" }</pre>	<pre>"Tracer": { "role": "Damage", "subrole": "Flanker" }</pre>
Dicționarul de abilități	Dicționarul de tipuri
<pre>"Tracer": { "Weapon": ["Pulse Pistols"], "Ability": ["Blink", "Recall"] }</pre>	<pre>"Pulse Pistols": "Weapon", "Blink": "Ability"</pre>

Tabelul 3.1: Cele 3 tipuri de dicționare comune și dicționarul de tipuri, specific Overwatch

3.1.2 Extracția datelor de pe Fandom

Modulul de extracție primește un URL către o pagină Fandom de tip *patch notes*, pe care o accesează prin intermediul API-ului MediaWiki pentru a obține text de tip *wikitext* (i.e. text formatat special pentru pagini Fandom), ce reprezintă întreg conținutul paginii. Textul este prelucrat linie cu linie pentru a fi curățat de artefacte de formatare, iar comentariile dezvoltatorilor explicit menționate sunt detectate și filtrate printr-un sistem de ancore, pentru a captura și acele comentarii ce se extind pe mai multe linii.

Textul rămas include secțiunile irelevante pentru pipeline, cum ar fi informații generale despre actualizarea respectivă, promoții limitate, moduri de joc noi, îmbunătățiri ale performan-

ței, evenimente sezonale etc. Prin definirea unor liste de cuvinte cheie, textul este scanat la nivelul subtitlurilor, și este redus la a conține doar *balance changes*. În implementarea pentru Overwatch, există în plus un mecanism ce caută și repară subtitluri cărora le lipsesc marcasele corespunzătoare. Deoarece în unele *patch notes* mai vechi se întâmplă ca editorii Fandom să nu folosească caracterele speciale pentru a defini subtitluri, sistemul caută cuvinte cheie izolate, aprobate, și le adaugă caracterele necesare pentru a putea fi recunoscute ca subtitluri mai târziu. De exemplu, dacă o linie conține exclusiv „Hero Changes”, aceasta este transformată în „== Hero Changes ==” pentru a fi identificată drept subtitlu de nivel doi la momentul procesării.

Deoarece Overwatch are două variante de joc pentru modul principal, 5v5 și 6v6, numite compoziții, iar unele schimbări se aplică doar uneia din ele, sistemul scanează liniile pentru a detecta și normaliza marcatorul de compoziție. În plus, deseori abilitățile sunt afișate împreună cu tipul acestora, astfel liniile cu abilități necesită o verificare și normalizare a prefixului și sufixului rămase după extragerea simbolurilor de formatare. Cel mai complex caz poate conține o abilitate, tipul acesteia, și marcatorul de compoziție, în ordine aleatoare, cu orice delimitatoare între ele, iar normalizarea le va aduce în ordinea menționată mai sus.

În final, liniile ce aparțin secțiunilor rămase sunt verificate pentru simboluri de indentare, pe măsura cărora este calculată adâncimea liniei respective, necesară pentru a putea afișa schimbările așa cum au fost destinate de către dezvoltatori. De exemplu, pentru o linie care conține la începutul ei două asteriscuri, adâncimea va fi doi.

Rezultatul este un dicționar cu data actualizării (extrasă direct din URL-ul sursă) și liniile destinate procesării, structurate la rândul lor într-o listă de dicționare conținând textul, adâncimea, și ordinea din textul sursă pentru fiecare linie, astfel:

```
1 {"date": "10.02.2026",  
2  "lines": [  
3    {"depth": 0, "source_order": 22, "text": "Tracer"},  
4    {"depth": 0, "source_order": 23, "text": "Pulse Pistols"},  
5    {"depth": 1, "source_order": 24,  
6    "text": "Damage decreased from 6 to 5.75"}]]}
```

3.1.3 Procesarea schimbărilor

Procesarea are rolul de a ruta schimbările ce nu sunt legate de entități, și de a prepara celelalte schimbări pentru o procesare mai amănunțită. Inițial, metadatele actualizării sunt înregistrate în baza de date orientată pe grafuri, după care toate liniile de text primite de la modulul de extracție sunt parsate secvențial. După validarea textului și adâncimii, fiecare linie este verificată pentru a actualiza secțiunea curentă.

La detectarea unui nou subtitlu (reprezentat printr-o linie de adâncime 0 încadrată de caractere specifice), toate indicatoarele de stare, ancorele de parsare, contextul curent, și subtitlurile inferioare sunt resetate. Pe baza noului set de subtitluri active, sunt calculate indicatoarele de rutare automată pentru acele blocuri de text care nu necesită o parsare amănunțită (e.g. actualizările generale ce au trecut prin filtrul din modulul de extracție, *bug fix*-urile). Cât timp aceste indicatoare rămân active, liniile subsecvente sunt salvate direct în baza de date, cu clasificarea dată de indicator.

Deoarece în unele *patch notes* există linii indentate prea mult, iar verificarea de context este făcută doar la adâncimea 0, sistemul se asigură că toate liniile au adâncimea corectă prin indicatoare de scădere a indentării, la nivel de secțiune, erou, sau abilitate. Aceste indicatoare sunt activate atunci când prima linie după un punct de control nu are adâncimea așteptată și sunt resetate atunci când se ajunge la un nou punct de control de același tip. Cât timp sunt active, indicatoarele normalizează adâncimea tuturor liniilor.

La pasul următor, liniile cu adâncimea 0 sunt scanate de modulul NER încărcat cu entitățile jocului respectiv, pentru a identifica modificări ale contextului. Deoarece schimbările sunt redactate frecvent sub formă de liste ierarhice pentru a minimiza redundanța textului, menținerea contextului este imperativă; în caz contrar, sistemul nu ar putea atribui corect schimbările. Dacă sunt doi eroi, încearcă să-i dedupliceze; dacă este un erou și o abilitate, verifică dacă eroul deține acea abilitate; dacă sunt două abilități unice, verifică dacă una din ele este cumva tipul celeilalte; dacă linia nu se încadrează în niciunul din cazuri, adâncimea îi este crescută, deoarece liniile ce ajung la acest nivel reprezintă schimbările efective.

La ultimul pas, contextul liniei este construit și alipit la începutul liniei, într-un format cât mai natural posibil, în încercarea de a face schimbarea să sune ca o propoziție completă, lucru

important pentru antrenarea modelului (mai multe detalii la secțiunea Antrenarea modelului). În final, linia este trimisă mai departe pentru clasificarea finală.

3.1.4 Analiza amănunțită

Modulului de analiză amănunțită folosește *en_core_web_sm*, un model spaCy [3] ușor, generalist, căruia îi este adăugat un *entity_ruler* personalizat, încărcat cu dicționarul de entități al jocului respectiv. Acest modul implementează funcția care scanează liniile și identifică entitățile din interiorul ei, folosită și în modulul de procesare a schimbărilor.

Codul de mai jos reprezintă încărcarea modulului NER și a dicționarului cu entități specific jocului, cu verificările și mesajele de eroare șterse pentru simplitate.

```
1 self.nlp = spacy.load("en_core_web_sm")
2 ruler = self.nlp.add_pipe("entity_ruler", before="ner",
   ↪ config={"phrase_matcher_attr": "LOWER"})
3 dict_path = os.path.join(self._game_dicts_dir,
   ↪ f"{self.game_slug}_dictionary.json")
4 patterns: List[Dict[str, str]] = []
5 if os.path.exists(dict_path):
6     with open(dict_path, "r", encoding="utf-8") as f:
7         patterns = json.load(f)
8 ruler.add_patterns(patterns)
```

În acest modul, liniile trec prin ultimul strat de procesare, analiză, și clasificare. Numai liniile legate de entități (i.e. cu context alipit) ajung în acest punct, și doar două clasificări (în afara de necunoscut) pot fi asiguate: schimbare mecanică, sau schimbare numerică, care la rândul său poate fi *buff*, sau *nerf*. Pentru a ilustra mecanismul, voi folosi ca exemplu următoarea linie: „Tracer’s Pulse Pistols - Weapon Damage decreased from 6 to 5.75”.

La început, sistemul extrage entitățile din linia primită. În caz că sunt detectați mai mulți eroi sau mai multe abilități, înseamnă că linia conține și alte entități în afara contextului alipit în modulul de procesare. Sistemul încearcă diverse tehnici pentru a reduce numărul rezultatelor la un erou și maxim o abilitate; printre acestea se numără deduplicarea, colapsarea (eliminarea abilităților secundare care aparțin eroului principal) sau unificarea (normalizarea abilităților secundare la tipuri ce se mapează abilității principale). De asemenea, dacă a detectat un erou și o abilitate, dar aceasta este de fapt un tip, atunci se încearcă înlocuirea acestuia cu abilitatea

corespunzătoare lui în lista de abilități a eroului. În cazul exemplului dat, sunt identificați un erou (Tracer) și două abilități (Pulse Pistols și Weapon). Unificarea este soluția necesară, dar tipul abilității nu este cel așteptat. Deoarece armele principale ale eroilor sunt deseori numite diferit, se aplică o normalizare a tipului, cu scopul de a aduce toate variantele la a fi ori *Primary Fire*, ori *Secondary Fire*. *Weapon* este folosit doar când eroul are o singură armă, deci este normalizat la *Primary Fire*, care este tipul așteptat de abilitatea *Pulse Pistols*, prin urmare *Weapon* este eliminat complet, linia devenind „Tracer’s Pulse Pistols Damage decreased from 6 to 5.75”.

Următorul pas este extragerea contextului, normalizarea abilității din context la forma canonică dacă forma curentă este o prescurtare sau un alias, și concentrarea pe analiza liniei. Dacă linia conține cuvinte cheie asociate schimbărilor numerice, sistemul încearcă să infereze un atribut. Cum schimbările numerice respectă un format strict, ori atribut + verb + secvență numerică, ori verb + atribut + secvență numerică, sistemul deduce atributul și verifică dacă este recunoscut. După aceea, atributul candidat este căutat dacă conține o abilitate, caz în care este extrasă, iar ce a rămas este verificat din nou în raport cu attributele cunoscute. Candidații care trec de aceste teste sunt salvate în baza de date relațională: un atribut fără abilitate, sau un atribut compus dintr-o abilitate combinată cu un atribut vechi. Dacă a fost dedus un atribut nou, un semnal este trimis pentru a se reconstrui modulul după finalizarea ingestiei actualizării, deoarece noul atribut trebuie să fie adăugat în *entity_ruler* pentru a putea fi identificat în viitoarele actualizări, fără a trece prin același mecanism de inferare de fiecare dată. În cazul exemplului dat, după extragerea contextului „Tracer’s Pulse Pistols”, linia devine „Damage decreased from 6 to 5.75”. Se observă că respectă formatul atribut + verb + secvență numerică, iar atributul găsit este „Damage”. Fiind un atribut comun, acesta este recunoscut, nefiind nevoie de o deducere.

După deducerea sau recunoașterea atributului, sistemul verifică dacă poate clasifica linia drept schimbare mecanică. Acestea sunt reprezentate de modificarea felului în care funcționează ceva, și sunt identificabile prin prezența unor cuvinte cheie mecanice (e.g. „no longer”, „now”, „only”). Pe lângă această condiție, este necesar ca schimbarea să nu conțină și cuvinte cheie numerice, și o secvență numerică, deoarece prezența amândurora poate însemna că de fapt linia este una dintre acele schimbări numerice ce conțin și un cuvânt cheie mecanic. Dacă a ajuns până în acest punct o linie cu multipli eroi și/sau abilități, singura opțiune pe care o mai are linia

pentru a nu fi clasificată ca necunoscută este să conțină o virgulă, semn că este o enumerare, deci o schimbare mecanică. În cazul exemplului dat, chiar dacă linia ar fi conținut un cuvânt cheie mecanic, tot nu ar fi fost clasificată astfel, deoarece conține și un cuvânt cheie numeric („decreased”), și o secvență numerică („from 6 to 5.75”).

După toate aceste verificări, normalizări și clasificări, dacă linia conține cuvinte cheie numerice, un număr de unul, două, patru, sau șase numere, o secvență numerică, și nu conține virgule, atunci a intrat în parsarea schimbărilor numerice. Cele 4 tipuri de secvențe numerice sunt par-sate, valoarea schimbării nete este calculată pe baza tipului de unitate (fixă sau procentuală), iar indicatorul de valoare este calculat pe baza semnului schimbării nete. Cum unele attribute beneficiază din scăderi, indicatorul este verificat și revizuit în raport cu o listă de attribute inversate. În final, valorile numerice sunt approximate la 3 decimale, și linia este clasificată ca schimbare *buff* sau schimbare *nerf*, în funcție de indicator. În cazul exemplului dat, schimbarea este de tip *nerf*, deoarece valoarea netă este negativă, iar atributul nu este inversat.

3.1.5 Integritatea informației

Integritatea datelor este în primul rând asigurată de strictețea procesării și a parsării. Euristici-cile au fost evitate aproape complet, singurele folosite fiind mai stricte decât era necesar tocmai pentru a evita introducerea de date greșite în bazele de date. Astfel, toate liniile pe care sistemul nu este sigur cum să le clasifice sunt plasate într-o coadă de verificări manuale, implementând un mecanism *human-in-the-loop*.

Operatorul uman poate să reformuleze textul, să adauge manual attribute, attribute inversate, cuvinte cheie numerice sau mecanice, sau mapări de prescurtări sau alias-uri, și apoi să folosească funcția de retrimiteră a liniei către analiza amănunțită. Dacă analiza reușește, linia este inserată în baza de date orientată pe grafuri exact unde îi este locul, și este marcată corespun-zător în baza de date relațională; dacă nu, linia rămâne neschimbată. Operatorul uman are și opțiunea să îi atribuie manual unei linii o clasificare. Această opțiune nu este recomandată decât dacă linia este într-adevar un caz special, când reformularea textului nu ajută iar adăugarea de elemente noi ar perturba clasificarea altor schimbări.

În algoritmul inferării atributelor, se poate să fie deduse attribute greșite, iar de aceea noile

attribute sunt salvate cu un indicator de sincronizare setat fals. Operatorul uman este nevoit să intre în interfața admin și să aprobe manual fiecare nou atribut, astfel declanșând un commit către locația codului sursă în GitHub, pentru a salva attributele în fișierul dedicat, unică sursă de adevăr. Dacă operatorul uman decide că atributul a fost greșit dedus, acesta declanșează un mecanism de ștergere a tuturor liniilor afectate, de plasare a acelor linii în coada de verificări manuale, și de ștergere a atributului. O altă funcție, mai generală, oferă operatorului uman posibilitatea să șteargă și să plaseze orice linie în coada de verificări, cu opțiunea de a șterge și atributul asociat acestora.

3.1.6 Studiu de caz: integrarea jocului Marvel Rivals

Pentru a demonstra modularitatea sistemului, am adăugat suport pentru Marvel Rivals. Urmând pașii din Anexa 1, am implementat funcțiile abstracte necesare și am suprascris metodele unde arhitectura jocului a impus o logică diferită, după cum urmează.

Modulul Builder: Procesul de obținere a abilităților a trebuit adaptat structurii specifice a platformei Fandom pentru Marvel Rivals. Spre deosebire de Overwatch, unde abilitățile erau extrase din marcare individuale, noul joc utilizează formatul de *SkillTables*. Astfel, funcția de parsare a fost regândită pentru a naviga și extrage corect entitățile din aceste tabele centralizate.

Modulul Extractor: O provocare a fost prezența comentariilor dezvoltatorilor care nu erau marcate prin ancorele oficiale (*dev comments*), ci erau doar încadrate de paranteze rotunde. Pentru a curăța datele de intrare, am suprascris funcția responsabilă de normalizarea intermediară, adăugând o regulă care elimină complet liniile delimitate integral de paranteze.

Modulul Parser: În procesul de parsare secvențială, un exemplu de problemă nerezolvată nativ de codul comun este faptul că, deseori, schimbările din actualizările Marvel Rivals nu au o relație 1-la-1 cu liniile textului. Unele linii conțin mai multe schimbări independente aplicate aceleiași abilități. Astfel, în cadrul funcției ce iterează prin text, am adăugat un mecanism ce împarte linia în propoziții, se asigură că toate propozițiile au aceeași abilitate în context sau fiecare are maxim o abilitate proprie, și le rulează individual. Pentru a prezerva ordinea sursei în baza de date, indicelui de ordine *i* se adaugă numere raționale:

¹ `if ". " in text:`

```

2 sentences = [s.strip() for s in text.split(". ") if s.strip()]
3 if len(sentences) > 1:
4     unique_abilities = list(dict.fromkeys(abilities))
5     if len(unique_abilities) > 1:
6         for s in sentences:
7             s_entities = nlp.extract_line_patterns(s)
8             s_unique_abilities =
9                 ↪ list(dict.fromkeys(s_entities.get("abilities",
10                 ↪ [])))
11             if len(s_unique_abilities) > 1:
12                 review_logged = self.log_manual_review( ... )
13                 self._set_autoroute_anchor(manual_review=depth)
14                 return review_logged
15 if len(unique_abilities) == 1:
16     self.current_ability = unique_abilities[0]
17 for i, t in enumerate(sentences, start=1):
18     split_line = line.copy()
19     split_line["text"] = t
20     split_line["source_order"] = (round(float(source_order) +
21     ↪ (i / 10.0), 1))
22     review_logged = self._process_line(game, patch_date,
23     ↪ patch_url, split_line, nlp, graph) or review_logged
24 self.current_ability = None
25 return review_logged

```

Modulul NLP: În cele din urmă, modulul de analiză amănunțită a necesitat o implementare standard, minimalistă. Singura modificare a fost o suprascriere a funcției de extragere a contextului, din cauza unor abilități problematice ale noului joc ce au necesitat identificare pe baza unui *id*.

3.2 Graful de cunoștințe

Graful de cunoștințe al unui joc este rezultatul ingestiei tuturor actualizărilor jocului, stocate în baza de date orientată pe grafuri. Deoarece scopul este obținerea valorilor nete ale modificărilor dintr-un interval de timp, graful este independent de valorile inițiale ale entităților jocului.

Arhitectura grafului este reprezentată de noduri. Fiecare joc, mod de joc, tip de compoziție, actualizare, erou, abilitate, atribut și schimbare are nodul său. Legăturile dintre ele sunt structurate în patru tipuri: aparține de, se aplică la, cauzează, și afectează. „Aparține de” este legătura dintre actualizare și joc, dintre erou și abilitate, dintre abilitate și atribut, și dintre erou și atribut. „Se aplică la” este legătura dintre schimbare și mod de joc, „cauzează” este legătura dintre

actualizare și schimbare, iar „afectează” este legătura dintre schimbare și atribut.

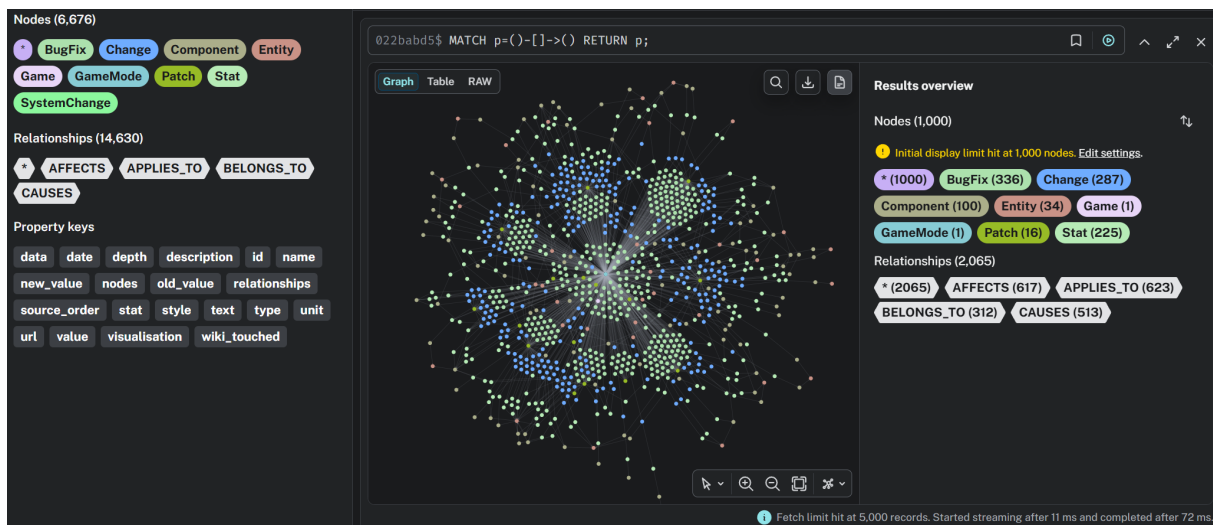


Figura 3.1: O parte a grafului de cunoștințe, vizualizat în interfața Neo4j

Un exemplu de schimbare numerică salvată în baza de date este următorul:

```
1 {"hero": "Tracer", "ability": "Pulse Pistols", "stat": "damage",
2  "type": "Nerf",
3  "text": "Tracer's Pulse Pistols Damage decreased from 6 to 5.75",
4  "value": -0.25, "old_value": 6, "new_value": 5.75, "unit": "flat"}
```

Interogarea datelor este facilitată astfel de arhitectura aleasă: când un utilizator dorește să vadă schimbările unui joc dintr-un interval de timp setat de acesta, informațiile sunt găsite rapid prin legarea jocului de actualizări, filtrarea actualizărilor în intervalul de timp, și legarea tuturor schimbărilor din acele actualizări, împreună cu entitățile cărora li se aplică modificările. Pentru a îmbunătăți și mai mult timpul de răspuns, au fost creați indecși pentru nodurile de tip actualizare, schimbare, mod de joc, erou, abilitate, și atribut.

Codul de mai jos reprezintă partea din interogare care se ocupă cu obținerea schimbărilor mecanice și numerice. Schimbările care nu sunt atașate unor entități sunt obținute separat, iar rezultatele sunt reunite folosind clauze *UNION ALL*.

```
1 MATCH (p:Patch)-[:BELONGS_TO]->(g:Game {name: $g})
2 WHERE p.date >= $start AND p.date <= $end
3 MATCH (p)-[:CAUSES]->(ch:Change)-[:APPLIES_TO]->(m:GameMode)
4 WHERE ch.type IN ["Buff", "Nerf", "Mechanical"]
```

```

5 MATCH (ch)-[:AFFECTS]->(s:Stat)
6 OPTIONAL MATCH (s)-[:BELONGS_TO]->(c:Component)
7 OPTIONAL MATCH (s)-[:BELONGS_TO]->(e_direct:Entity)
8 OPTIONAL MATCH (c)-[:BELONGS_TO]->(e_component:Entity)
9 RETURN m.name AS gamemode, p.date AS patch_date,
    ↪ coalesce(e_component.name, e_direct.name, "Unknown") AS entity,
    ↪ coalesce(c.name, "None") AS component, s.name AS stat, ch.text AS
    ↪ text, ch.depth AS depth, coalesce(ch.source_order, 2147483647) AS
    ↪ source_order, ch.type AS type, ch.value AS value, ch.old_value AS
    ↪ old_value, ch.new_value AS new_value, ch.unit AS unit
10 ORDER BY patch_date DESC, source_order ASC, text ASC

```

3.3 Aplicația web

Aplicația web construită pe baza sistemului reprezintă o unealtă folosită de jucătorii care revin dintr-o pauză îndelungată, deoarece minimizează timpul petrecut citind și analizând actualizările jocului, activitate ce devine cu atât mai obositoare cu cât pauza este mai mare.

După ce utilizatorul selectează jocul dorit în prima pagină (Figura 3.2), acesta este întâmpinat cu o interfață cât mai similară cu site-ul web oficial al acelui joc, pentru o tranziție cât mai lină (Figura 3.3 și Figura 3.4).

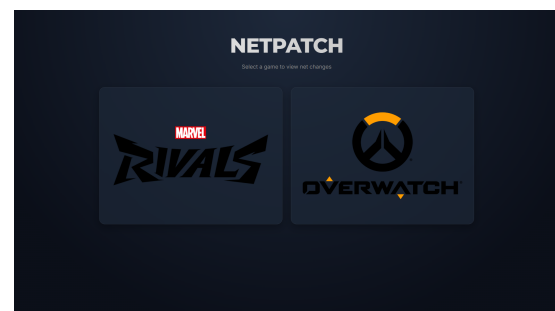


Figura 3.2: Prima pagină a aplicației

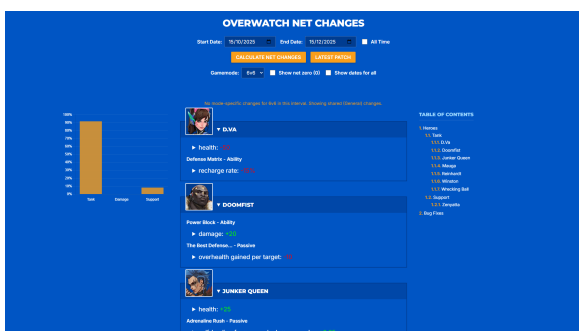


Figura 3.3: Pagina Overwatch a aplicației web NetPatch

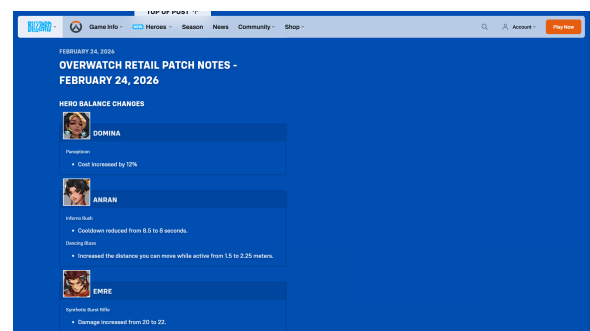


Figura 3.4: Pagina de patch notes a site-ului oficial Overwatch

Framework-ul Django integrează nativ o interfață de administrare care facilitează controlul

direct asupra datelor salvate în PostgreSQL. Din acest panou, operatorul uman poate revizui liniile aflate în coada de verificări manuale, adăuga noi cuvinte cheie, aproba atributele deduse, reiniția analiza unor linii selectate și exporta seturile de text necesare antrenării modelului.

3.4 Modulul NER pentru Overwatch

Când o linie este clasificată ca fiind mecanică sau numerică, aceasta este stocată în baza de date relațională. Liniile sunt salvate cu contextul alipit, iar entitățile detectate în interiorul lor sunt cele înregistrate cu ajutorul dicționarelor în *entity_ruler*. Astfel, pentru Overwatch, după ingestia tuturor actualizărilor am obținut un set de date de 1979 de linii adnotate automat, din 141 de actualizări, de pe 04.10.2022 și până pe 12.05.2026.

3.4.1 Antrenarea modelului

Prin exportarea tuturor liniilor aparținând actualizărilor de dinainte de 2026 am obținut un set de date însumând 1818 linii. Acesta este folosit pentru antrenare, iar liniile rămase, din anul 2026, sunt păstrate pentru a evalua modelul în raport cu date nemaivăzuite până la momentul respectiv. Din cele 1818 linii rezervate pentru antrenare, doar 80% din ele sunt folosite pentru antrenarea efectivă, restul de 20% fiind alocate pentru validare în timpul epocilor, pentru a calcula metricile intermediare și pentru a salva cel mai bun model.

După deduplicare pentru a preveni overfitting, cele două seturi de date sunt convertite din formatul jsonl în formatul binar spacy menit pentru antrenare. Fiecare linie este tokenizată conform regulilor standard pentru limba engleză, iar indicii de caractere sunt transformați în indicii de tokeni. În caz de suprapunere, este aplicat un mecanism de păstrare a celor mai lungi entități.

Pentru a defini arhitectura rețelei neuronale, nu a fost necesară construirea manuală a structurilor, ci declararea constrângerilor de performanță printr-un fișier de configurare standardizat, generat de spaCy. Parametrul `--optimize accuracy` semnalizează ca timpul de execuție sau consumul de memorie să fie ignorate, iar parametrul `--gpu` aduce sistemului la cunoștință că deține infrastructură necesară pentru a antrena un model masiv. Mai exact, framework-

ul a selectat și configurat automat un model de limbaj masiv pre-antrenat de tip Transformer (RoBERTa) pe post de extractor de context. RoBERTa procesează propoziția și transformă cuvintele în reprezentări numerice dense, numite și stări ascunse, care captează sensul și relațiile gramaticale. Aceste date sunt apoi transmise mai departe către o componentă de clasificare ce analizează contextul primit și aplică etichetele pe text. Această arhitectură a fost preferată nativ de sistem deoarece oferă reprezentări contextuale superioare, folositoare pentru descifrarea jargonului. Comanda utilizată pentru generarea acestei configurații este:

```
1 python -m spacy init config config.cfg --lang en --pipeline ner
   ↪ --optimize accuracy --gpu
```

În final, este apelată comanda de antrenare ce se folosește de fișierul de configurare generat anterior, împreună cu cele două fișiere de tip spacy ce reprezintă setul de antrenare și setul de validare:

```
1 python -m spacy train config.cfg --output ./trained_model
   ↪ --paths.train ./data/train.spacy --paths.dev ./data/dev.spacy
   ↪ --gpu-id 0
```

Pentru evaluarea modelului, sunt folosite restul de 161 de linii aparținând de actualizările din 2026. Setul de date este trecut prin aceleași mecanisme ca și cele două seturi de antrenare și validare: deduplicare, tokenizare a textului, transformarea indicilor în tokeni. Următoarea comandă este rulată pentru a obține rezultatele:

```
1 python -m spacy evaluate ./trained_model/model-best
   ↪ ./data/test_2026.spacy --gpu-id 0
```

Rezultatele obținute în urma evaluării sunt centralizate în Tabelul 3.2. Acestea reflectă performanța modelului pe cele trei categorii de entități recunoscute, exprimate prin Precizie (P), Recall (R) și Scorul F1 (F). Precizia măsoară proporția predicțiilor corecte din totalul entităților semnalate de model, penalizând astfel fals-pozitivele, Recall reprezintă proporția entităților reale (prezente în text) pe care modelul a reușit să le găsească, penalizând astfel fals-negativele, iar Scorul F1 constituie media armonică a celor două.

Entitate	Precizie (%)	Recall (%)	Scor F1 (%)
EROU	90.15	84.72	87.35
ABILITATE	84.38	92.57	88.28
ATRIBUT	75.25	75.25	75.25

Tabelul 3.2: Rezultatele evaluării modelului NER pe setul de date din 2026

Pentru a demonstra capacitatea de generalizare, Tabelul 3.3 include o comparație între scorul F1 obținut pe setul de validare (extras din perioada 2022-2025) și performanța detaliată pe setul de testare (anul 2026).

Entitate	Validare (2022-2025) F1 (%)	Testare (2026) F1 (%)
EROU	99.87	87.35
ABILITATE	97.13	88.28
ATRIBUT	95.93	75.25

Tabelul 3.3: Comparația scorurilor F1 între setul de validare și setul de testare

Scorurile obținute pe setul de validare sunt extrem de ridicate (peste 95% pentru toate categoriile), indicând faptul că rețeaua a asimilat eficient structura datelor de antrenare. Scăderea performanței pe setul de testare (aproximativ 87-88% Scor F1 pentru recunoașterea eroilor și a abilităților) demonstrează că modelul Transformer nu a suferit de overfitting memorând pur și simplu intrările, ci a învățat contextul structural, fiind capabil să generalizeze pe date noi adnotate automat.

Scorul mai scăzut obținut pentru atribute (75.25%) este explicabil prin felul în care atributele sunt deduse în timpul execuției din structura schimbărilor. Acest mecanism duce inevitabil la introducerea unor atribute greșite în baza de date. Intervenția viitoare a unui operator uman în validarea tuturor liniilor acceptate și direcționarea celor cu atribute inferate greșit către coada de verificări va crește cu siguranță acuratețea setului de date, și implicit a modelului. Cu toate acestea, performanța globală este net superioară alternativelor bazate exclusiv pe reguli statice.

3.4.2 Modulul NER instalabil

Pentru a transforma acest model dintr-un simplu artefact local într-un produs software reutilizabil, folderul cu cele mai bune ponderi (model-best) a fost compilat într-un pachet instalabil. Rezultatul a fost arhivat sub forma unui modul standard Python (en_overwatch_ner-1.0.0.tar.gz) folosind următoarea comandă de împachetare spaCy:

```
1 python -m spacy package ./trained_model/model-best ./packages --name  
   ↪ overwatch_ner --version 1.0.0
```

Astfel este abstractizată complet complexitatea arhitecturală a modelului RoBERTa și a regulilor de tokenizare. Modulul NER rezultat se comportă ca o bibliotecă nativă, putând fi distribuit și instalat în orice mediu virtual prin intermediul managerului de pachete pip și importat într-o aplicație externă cu o singură linie de cod. Astfel, oferă o metodă directă și rapidă de a extrage entitățile dintr-un text brut specific jocului Overwatch.

4. Concluzii

Lucrarea abordează problema extragerii, cuantificării și structurării informațiilor din notele de actualizare ale jocurilor video. Rezultatul principal este platforma NetPatch, un sistem hibrid complet funcțional care demonstrează viabilitatea aplicării tehnicilor de procesare a limbajului natural (NLP) pe date nestructurate din domeniul jocurilor video.

Prin implementarea acestui sistem, au fost obținute mai multe rezultate derivate cu o utilitate practică ridicată.

- Graful de cunoștințe ce mapează eficient relațiile dintre actualizări, entități și modificările numerice sau mecanice suferite de acestea.
- Aplicația web ce oferă jucătorilor o interfață vizuală intuitivă pentru accesarea rapidă a schimbărilor nete, eliminând efortul parcurgerii manuale a unui istoric vast.
- Modulul NER (bazat pe arhitectura RoBERTa) dedicat jocului Overwatch, dezvoltat prin mecanismul de adnotare automată ce a permis generarea unui set de date specific.

În ciuda rezultatelor obținute, sistemul prezintă o serie de limitări inerente arhitecturii și sursei de date. Calitatea datelor extrase depinde de consecvența editorilor platformei Fandom: erorile umane de formatare sau lipsa standardizării marcajelor de tip *wikitext* pot îngreuna mecanismul de parsare, necesitând filtre de normalizare complexe.

O altă limitare decurge din utilizarea metodei supervizării la distanță pentru adnotarea automată a setului de date. Mecanismul care inferă atributele noi direct din structura textului este eficient pentru scalabilitate, însă introduce inevitabil un anumit nivel de „zgomot” în baza de date sub forma unor atribute deduse greșit. Această limitare s-a reflectat direct în evaluarea modelului NER pe setul de testare, unde acuratețea recunoașterii atributelor a înregistrat un scor F1 de 75.25%, vizibil mai mic în comparație cu performanța obținută la recunoașterea eroilor și a abilităților.

Pentru a adresa limitările identificate și a extinde impactul proiectului, direcțiile viitoare de dezvoltare se vor concentra pe rafinarea datelor și pe distribuția modulului creat.

Primul obiectiv vizează utilizarea sistemului de revizuire manuală (*human-in-the-loop*) deja integrat în platformă. Deși mecanismele de revizuire, respingere, modificare și reanalizare a liniilor sunt complet funcționale în interfața de administrare, este necesar un efort de verificare și etichetare umană pentru a parcurge cele aproximativ 2000 de linii aprobate automat în faza de ingestie, precum și cele 800 de linii aflate în coada de așteptare. Finalizarea acestui efort de curățare a datelor va crește substanțial calitatea adnotărilor, permițând reantrenarea modelului NER și remedierea directă a acurateței scăzute înregistrate la recunoașterea atributelor.

Al doilea obiectiv vizează publicarea modulului *en_overwatch_ner*, compilat în prezent sub forma unui pachet local, pe platforma oficială PyPI (*Python Package Index*). Astfel, orice dezvoltator va putea să integreze recunoașterea entităților din Overwatch în propriile aplicații printr-o simplă linie de cod. După finalizarea etapei de reantrenare, modelul va fi republicat sub o nouă versiune.

Ulterior, pe măsură ce aplicația NetPatch va asimila noi jocuri, sistemul va putea genera și publica automat module NER dedicate pentru fiecare joc, formând astfel o suită de unelte *open-source* de înaltă precizie pentru analiza datelor din industria jocurilor video.

Bibliografie

- [1] Django Software Foundation, *Django Web Framework*, 2026, URL: <https://www.djangoproject.com/>.
- [2] Fandom, *MediaWiki API Documentation*, 2026, URL: https://www.mediawiki.org/wiki/API:Main_page (accesat în 7.3.2026).
- [3] Ines Montani, Matthew Honnibal, Adriane Boyd, Sofie Van Landeghem și Henning Peters, *explosion/spaCy: v3.7.2*, Oct. 2023, DOI: [10.5281/zenodo.10009823](https://doi.org/10.5281/zenodo.10009823).
- [4] Neo4j, Inc., *Neo4j Graph Database Platform*, 2026, URL: <https://neo4j.com/>.
- [5] Sergiu Nisioi, *rolegal: A spaCy Package for Romanian Legal Document Processing*, 2023, URL: <https://github.com/senisioi/rolegal> (accesat în 18.2.2026).
- [6] PostgreSQL Global Development Group, *PostgreSQL Relational Database Management System*, 2026, URL: <https://www.postgresql.org/>.

Anexa 1. Ghid de integrare a unui joc

Acest ghid detaliază pașii tehnici necesari extinderii platformei NetPatch pentru un nou joc, completând specificațiile arhitecturale prezentate în Secțiunea 3.1. La final, voi demonstra modularitatea sistemului prin detalierea implementării unui joc adițional, Marvel Rivals.

Inițial, colaboratorul va accesa pagina de GitHub a aplicației și va face *fork* repository-ului.

În cele ce urmează, voi folosi `jocnou` ca înlocuitor pentru numele noului joc, având toate literele mici și neavând niciun spațiu sau delimitator.

Următoarele fișiere și foldere trebuie să fie create (* reprezintă obligatoriu):

- `netpatch/tracker/services/jocnou/ *`
- `netpatch/tracker/services/jocnou/jocnou_builder.py *`
- `netpatch/tracker/services/jocnou/jocnou_extractor.py *`
- `netpatch/tracker/services/jocnou/jocnou_parser.py *`
- `netpatch/tracker/services/jocnou/jocnou_nlp.py *`
- `netpatch/tracker/fixtures/jocnou/ *`
- `netpatch/tracker/fixtures/jocnou/jocnou_general_data.json *`
- `netpatch/tracker/static/tracker/images/jocnou/`
- `netpatch/tracker/static/tracker/images/jocnou/heroes/`
- `netpatch/tracker/templates/jocnou.html *`
- `netpatch/tracker/tests/jocnou/`
- `netpatch/tracker/tests/jocnou/__init__.py`
- `netpatch/tracker/tests/jocnou/test_nlp.py`
- `netpatch/tracker/tests/jocnou/test_pipeline.py`

- netpatch/tracker/tests/jocnou/fixtures/
- netpatch/tracker/tests/jocnou/fixtures/expected_YYYYMMDD.txt

Fiecare fișier din folderul de servicii implementează o clasă ce moștenește clasele fișierelor omoloage din folderul netpatch/tracker/services/common/. În aceste clase, funcțiile abstracte trebuie implementate conform contractului, iar orice alte funcții pot fi extinse și adaptate noului joc.

Fișierul jocnou_general_data.json trebuie să includă cel puțin următoarele chei:

- game_prefix: reprezintă prescurtarea lui jocnou
- patch_cutoff_date: reprezintă data de la care sunt extrase actualizările
- extractor_include_keywords: reprezintă secțiunile incluse explicit în extractor
- extractor_exclude_keywords: reprezintă secțiunile excluse explicit în extractor

În folderul netpatch/tracker/static/tracker/images/jocnou/heroes/ se vor crea atâtea imagini de tip png câți eroi conține jocnou.

Fișierul jocnou.html va conține interfața grafică. Este de datoria colaboratorului să îl stilizeze cât mai similar cu pagina de *patch notes* a site-ului oficial jocnou.

În folderul netpatch/tracker/tests/jocnou, fișierul __init__.py va fi lăsat complet gol. În test_nlp.py se pot scrie oricâte teste de NLP (analiză amănunțită). În fișierul test_pipeline.py se pot scrie oricâte teste de pipeline, care verifică logurile unei actualizări contra conținutului așteptat. Pentru fiecare test de actualizare, trebuie creat fișierul expected_YYYYMMDD.txt aferent acesteia (unde YYYYMMDD se înlocuiește cu data actualizării).

După ce colaboratorul publică implementările necesare pe copia repository-ului de pe profilul său, acesta va deschide un *pull request* pentru verificare. După aprobare, implementarea dezvoltatorului este integrată în repository-ul oficial, iar dezvoltatorul va apărea printre colaboratori.